Event dashboard > Spec Coding > Infrastructure > Deployment Pipeline

Deployment Pipeline

A robust deployment pipeline ensures reliable, secure, and automated delivery of your MyCoolChatbot-KiroSpec application. This section creates comprehensive CI/CD specifications with quality gates, environment management, and deployment strategies.

Exercise 3.1: CI/CD Pipeline Architecture

Design comprehensive CI/CD pipeline using AWS native services:

~~~
Create detailed CI/CD pipeline specifications for our MyCoolChatbot-KiroSpec application:

**Pipeline Architecture Overview**:
Use the AWS documentation MCP server to research current CI/CD best practices, then design:

**Source Stage**:
- **Repository**: GitHub with branch protection rules
- **Branching Strategy**: GitFlow with feature, develop, release, and main branches
- **Webhook Integration**: Automatic pipeline triggers on push and pull requests
- **Code Scanning**: GitHub Advanced Security with CodeQL analysis

**Build Stage - Frontend**:
- **Build Environment**: AWS CodeBuild with Node.js 18 runtime
- **Build Specification**:
```yaml
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 18
    commands:
      - npm ci
  pre_build:
    commands:
      - npm run lint
      - npm run type-check
      - npm audit --audit-level moderate
  build:
    commands:
      - npm run build
      - npm run test:coverage
  post_build:
    commands:
      - aws s3 sync dist/ s3://$FRONTEND_BUCKET --delete
      - aws cloudfront create-invalidation --distribution-id $DISTRIBUTION_ID --paths "/*"
artifacts:
  files:
    - '**/*'
  base-directory: dist
```

**Build Stage - Backend**:
- **Build Environment**: AWS CodeBuild with Node.js 18 runtime
- **Lambda Packaging**: Automated dependency installation and packaging
- **Build Specification**:
```yaml
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 18
    commands:
      - npm ci
  pre_build:
    commands:
      - npm run lint
      - npm run type-check
      - npm run test:unit
      - npm audit --audit-level moderate
  build:
    commands:
      - npm run build
      - npm run package
  post_build:
    commands:
      - aws lambda update-function-code --function-name $FUNCTION_NAME --zip-file fileb://dist/function.zip
artifacts:
~~~

~~~
  files:
    - 'dist/**/*'
```

**Infrastructure Stage**:
- **Infrastructure as Code**: AWS CDK with TypeScript
- **Deployment Tool**: AWS CodeDeploy with CloudFormation
- **Build Specification**:
```yaml
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 18
    commands:
      - npm install -g aws-cdk
      - npm ci
  pre_build:
    commands:
      - cdk synth
      - npm run test:infrastructure
  build:
    commands:
      - cdk deploy --require-approval never --outputs-file outputs.json
  post_build:
    commands:
      - aws s3 cp outputs.json s3://$ARTIFACTS_BUCKET/outputs/
artifacts:
  files:
    - 'cdk.out/**/*'
    - 'outputs.json'
```

**Quality Gates**:
- **Code Quality**: SonarQube analysis with quality gates
- **Security Scanning**: SAST and dependency vulnerability scanning
- **Test Coverage**: Minimum 80% code coverage requirement
- **Performance Testing**: Automated performance benchmarks
- **Infrastructure Validation**: CDK synthesis and security checks

Research current AWS CodePipeline best practices and integration patterns using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive CI/CD pipeline architecture with AWS native services
- Detailed build specifications for frontend, backend, and infrastructure
- Quality gates ensuring code quality, security, and performance
- Automated testing and validation at each stage
- Integration with source control and artifact management

Exercise 3.2: Environment Management Strategy

Create comprehensive environment management and promotion strategies:

~~~
Design detailed environment management specifications for our chatbot deployment:

**Environment Definitions**:

**Development Environment**:
- **Purpose**: Individual developer testing and feature development
- **Infrastructure**: Minimal resources for cost optimization
- **Configuration**:
```json
{
  "environment": "dev",
  "lambdaMemory": 256,
  "lambdaConcurrency": 10,
  "apiGatewayThrottling": {
    "rateLimit": 100,
    "burstLimit": 200
  },
  "dynamodbBilling": "PAY_PER_REQUEST",
  "cloudFrontPriceClass": "PriceClass_100",
  "bedrockModel": "anthropic.claude-3-haiku-20240307-v1:0",
  "logLevel": "DEBUG",
  "monitoring": "basic"
}
```

**Testing/QA Environment**:
- **Purpose**: Integration testing, QA validation, and staging
- **Infrastructure**: Production-like configuration with reduced scale
- **Configuration**:
~~~

~~~~
```json
{
  "environment": "test",
  "lambdaMemory": 512,
  "lambdaConcurrency": 50,
  "apiGatewayThrottling": {
    "rateLimit": 500,
    "burstLimit": 1000
  },
  "dynamodbBilling": "PROVISIONED",
  "cloudFrontPriceClass": "PriceClass_200",
  "bedrockModel": "anthropic.claude-3-haiku-20240307-v1:0",
  "logLevel": "INFO",
  "monitoring": "enhanced"
}
```

**Staging Environment**:
- **Purpose**: Pre-production validation and performance testing
- **Infrastructure**: Production-identical configuration
- **Configuration**:
```json
{
  "environment": "staging",
  "lambdaMemory": 1024,
  "lambdaConcurrency": 100,
  "apiGatewayThrottling": {
    "rateLimit": 1000,
    "burstLimit": 2000
  },
  "dynamodbBilling": "PROVISIONED",
  "cloudFrontPriceClass": "PriceClass_All",
  "bedrockModel": "anthropic.claude-3-sonnet-20240229-v1:0",
  "logLevel": "WARN",
  "monitoring": "comprehensive"
}
```

**Production Environment**:
- **Purpose**: Live customer-facing application
- **Infrastructure**: Optimized for performance, availability, and cost
- **Configuration**:
```json
{
  "environment": "prod",
  "lambdaMemory": 1024,
  "lambdaConcurrency": 1000,
  "apiGatewayThrottling": {
    "rateLimit": 2000,
    "burstLimit": 5000
  },
  "dynamodbBilling": "PROVISIONED",
  "cloudFrontPriceClass": "PriceClass_All",
  "bedrockModel": "anthropic.claude-3-sonnet-20240229-v1:0",
  "logLevel": "ERROR",
  "monitoring": "comprehensive"
}
```

**Environment Promotion Strategy**:
- **Automatic Promotion**: Dev → Test (on successful build)
- **Manual Approval**: Test → Staging (QA sign-off required)
- **Scheduled Promotion**: Staging → Production (maintenance windows)
- **Rollback Capability**: Immediate rollback to previous version
- **Blue/Green Deployment**: Zero-downtime production deployments

**Configuration Management**:
- **Parameter Store**: Environment-specific configuration parameters
- **Secrets Manager**: Environment-specific secrets and credentials
- **Feature Flags**: Environment-specific feature toggles
- **Environment Variables**: Runtime configuration for Lambda functions
- **Infrastructure Parameters**: CDK context and stack parameters

**Data Management Between Environments**:
- **Test Data**: Synthetic data generation for non-production environments
- **Data Masking**: PII anonymization for lower environments
- **Database Seeding**: Automated test data population
- **Data Isolation**: Complete separation between environments
- **Backup Strategy**: Environment-specific backup and retention policies

Research current environment management best practices and AWS parameter management using the AWS documentation MCP server.
~~~~

Expected Outcome

- Comprehensive environment definitions with appropriate resource sizing
- Clear promotion strategy with approval gates and rollback capabilities

- Configuration management strategy for environment-specific settings
- Data management and isolation between environments
- Cost optimization through environment-appropriate resource allocation

Exercise 3.3: Deployment Strategies and Automation

Design advanced deployment strategies for zero-downtime deployments:

~~~
Create detailed deployment strategy specifications for our MyCoolChatbot-KiroSpec application:

**Blue/Green Deployment Strategy**:

**Infrastructure Setup**:
- **Blue Environment**: Current production environment
- **Green Environment**: New version deployment target
- **Load Balancer**: CloudFront with origin switching capability
- **Database**: Shared between environments with migration strategy
- **Monitoring**: Parallel monitoring for both environments

**Deployment Process**:
```yaml
BlueGreenDeployment:
  phases:
    - name: "Prepare Green Environment"
      actions:
        - "Deploy infrastructure to green environment"
        - "Deploy application code to green environment"
        - "Run smoke tests on green environment"
        - "Validate database migrations"

    - name: "Traffic Switching"
      actions:
        - "Switch 10% traffic to green environment"
        - "Monitor metrics for 10 minutes"
        - "Switch 50% traffic to green environment"
        - "Monitor metrics for 10 minutes"
        - "Switch 100% traffic to green environment"

    - name: "Validation and Cleanup"
      actions:
        - "Run full test suite on green environment"
        - "Monitor production metrics for 30 minutes"
        - "Decommission blue environment"
        - "Update DNS records if needed"
```

**Canary Deployment Strategy**:
- **Canary Group**: 5% of users receive new version
- **Monitoring Period**: 30 minutes of metrics collection
- **Success Criteria**: Error rate < 0.1%, latency < 2s, user satisfaction > 95%
- **Automatic Rollback**: Triggered by metric thresholds
- **Progressive Rollout**: 5% → 25% → 50% → 100% traffic

**Lambda Deployment Configuration**:
- **Deployment Type**: AWS CodeDeploy with Lambda hooks
- **Alias Management**: Weighted routing between versions
- **Pre-traffic Hook**: Validation function execution
- **Post-traffic Hook**: Cleanup and monitoring setup
- **Rollback Configuration**: Automatic rollback on CloudWatch alarms

**Database Migration Strategy**:
- **Backward Compatible**: All migrations must be backward compatible
- **Migration Validation**: Automated testing of migration scripts
- **Rollback Scripts**: Automated rollback for failed migrations
- **Data Integrity**: Checksums and validation after migrations
- **Performance Impact**: Migration execution during low-traffic periods

**Rollback Procedures**:
- **Automatic Rollback Triggers**:
  - Error rate > 1% for 5 minutes
  - Average latency > 5 seconds for 5 minutes
  - Health check failures > 10% for 2 minutes
  - Custom business metric thresholds
- **Manual Rollback**: One-click rollback via AWS Console or CLI
- **Rollback Validation**: Automated testing after rollback completion
- **Communication**: Automated notifications during rollback events

**Deployment Automation**:
```yaml
DeploymentAutomation:
  triggers:
    - "Git tag creation (production)"
    - "Pull request merge (staging)"
    - "Scheduled deployment windows"

  validations:
~~~

~~~
    - "Infrastructure drift detection"
    - "Security vulnerability scanning"
    - "Performance regression testing"
    - "Integration test execution"

  approvals:
    - "Automated approval for dev/test"
    - "Manual approval for staging/production"
    - "Security team approval for infrastructure changes"
    - "Business stakeholder approval for major releases"
```

Research current deployment automation best practices and AWS CodeDeploy strategies using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive blue/green deployment strategy with traffic switching
- Canary deployment configuration with automated rollback
- Lambda-specific deployment strategies with alias management
- Database migration strategy with rollback capabilities
- Automated deployment processes with appropriate approval gates

Exercise 3.4: Quality Assurance and Testing Integration

Integrate comprehensive testing and quality assurance into the deployment pipeline:

~~~
Create detailed testing and QA specifications for our deployment pipeline:

**Automated Testing Strategy**:

**Unit Testing**:
- **Frontend Tests**: Jest with React Testing Library
- **Backend Tests**: Jest with supertest for API testing
- **Infrastructure Tests**: CDK unit tests with AWS CDK assertions
- **Coverage Requirements**: Minimum 80% code coverage
- **Test Execution**: Parallel execution for faster feedback

**Integration Testing**:
- **API Integration**: Automated API endpoint testing
- **Database Integration**: DynamoDB integration tests with local DynamoDB
- **AWS Service Integration**: Mocked AWS service interactions
- **End-to-End Flows**: Complete user journey testing
- **Contract Testing**: API contract validation between services

**End-to-End Testing**:
- **Testing Framework**: Playwright for cross-browser testing
- **Test Scenarios**:
```yaml
E2ETestScenarios:
  - name: "User Registration and Login"
    steps:
      - "Navigate to application"
      - "Register new user account"
      - "Verify email confirmation"
      - "Login with credentials"
      - "Verify dashboard access"

  - name: "Chat Functionality"
    steps:
      - "Login as existing user"
      - "Send chat message"
      - "Verify AI response received"
      - "Verify conversation history"
      - "Test message persistence"

  - name: "Error Handling"
    steps:
      - "Test network failure scenarios"
      - "Test invalid input handling"
      - "Test authentication failures"
      - "Verify error message display"
```

**Performance Testing**:
- **Load Testing**: Artillery.js for API load testing
- **Stress Testing**: Gradual load increase to breaking point
- **Spike Testing**: Sudden traffic spike simulation
- **Endurance Testing**: Extended load over time
- **Performance Benchmarks**:
  - API response time < 500ms (95th percentile)
  - Frontend load time < 2 seconds
  - Concurrent users > 1000 without degradation
  - Database query time < 100ms (average)
~~~

~~~
**Security Testing**:
- **SAST (Static Analysis)**: SonarQube with security rules
- **DAST (Dynamic Analysis)**: OWASP ZAP automated scanning
- **Dependency Scanning**: npm audit and Snyk integration
- **Infrastructure Security**: Checkov for IaC security scanning
- **Penetration Testing**: Quarterly external security assessments

**Quality Gates Configuration**:
```yaml
QualityGates:
  CodeQuality:
    - "Code coverage >= 80%"
    - "No critical SonarQube issues"
    - "No high-severity security vulnerabilities"
    - "All unit tests passing"

  Integration:
    - "All integration tests passing"
    - "API contract tests passing"
    - "Database migration tests passing"
    - "AWS service integration tests passing"

  Performance:
    - "Load test results within benchmarks"
    - "No performance regression > 10%"
    - "Memory usage within limits"
    - "Database performance acceptable"

  Security:
    - "No critical security vulnerabilities"
    - "SAST scan passing"
    - "DAST scan passing"
    - "Infrastructure security scan passing"
```

**Test Data Management**:
- **Test Data Generation**: Automated synthetic data creation
- **Data Anonymization**: PII removal for test environments
- **Test Data Cleanup**: Automated cleanup after test execution
- **Data Versioning**: Test data version control and management
- **Environment Isolation**: Separate test data for each environment

**Reporting and Metrics**:
- **Test Results**: Automated test result reporting and trending
- **Quality Metrics**: Code quality metrics and historical trends
- **Performance Metrics**: Performance test results and comparisons
- **Security Metrics**: Security scan results and vulnerability trends
- **Dashboard Integration**: Real-time quality dashboard with alerts

Research current testing automation best practices and AWS testing services using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive automated testing strategy covering all application layers
- Quality gates ensuring code quality, performance, and security
- Performance testing with clear benchmarks and regression detection
- Security testing integrated throughout the pipeline
- Test data management and reporting for continuous improvement

Exercise 3.5: Monitoring and Observability in CI/CD

Integrate comprehensive monitoring and observability into the deployment pipeline:

````
Create detailed monitoring and observability specifications for our deployment pipeline:

**Pipeline Monitoring**:

**Build and Deployment Metrics**:
- **Build Success Rate**: Percentage of successful builds over time
- **Build Duration**: Average and 95th percentile build times
- **Deployment Frequency**: Number of deployments per day/week
- **Lead Time**: Time from commit to production deployment
- **Mean Time to Recovery (MTTR)**: Time to recover from failed deployments

**Quality Metrics**:
- **Test Success Rate**: Percentage of passing tests by category
- **Code Coverage Trends**: Coverage percentage over time
- **Security Vulnerability Trends**: Number and severity of vulnerabilities
- **Performance Regression**: Performance metrics compared to baseline
- **Defect Escape Rate**: Production issues not caught by testing

**Deployment Health Monitoring**:
````

~~~
```yaml
DeploymentHealthChecks:
  PreDeployment:
    - "Infrastructure health validation"
    - "Dependency service availability"
    - "Database connectivity and performance"
    - "External service integration status"

  PostDeployment:
    - "Application health endpoint validation"
    - "Critical user journey execution"
    - "Performance benchmark validation"
    - "Security configuration verification"

  ContinuousMonitoring:
    - "Error rate monitoring (< 0.1%)"
    - "Response time monitoring (< 2s)"
    - "Throughput monitoring (baseline comparison)"
    - "Resource utilization monitoring"
```

**Alerting and Notification**:
- **Pipeline Failures**: Immediate notification to development team
- **Quality Gate Failures**: Notification with detailed failure information
- **Deployment Issues**: Escalation to on-call engineer
- **Security Vulnerabilities**: Immediate notification to security team
- **Performance Degradation**: Automated rollback trigger and notification

**Observability Integration**:
- **Distributed Tracing**: AWS X-Ray integration for request tracing
- **Log Aggregation**: CloudWatch Logs with structured logging
- **Metrics Collection**: Custom metrics for business and technical KPIs
- **Dashboard Creation**: Real-time dashboards for pipeline and application health
- **Anomaly Detection**: Machine learning-based anomaly detection

**Deployment Tracking and Audit**:
- **Deployment History**: Complete history of all deployments with metadata
- **Change Tracking**: Git commit to deployment traceability
- **Approval Audit**: Record of all manual approvals and reviewers
- **Rollback History**: Complete history of rollbacks with reasons
- **Compliance Reporting**: Automated compliance reports for audits

**Feedback Loops**:
- **Developer Feedback**: Fast feedback on code quality and test results
- **Operations Feedback**: Production metrics fed back to development
- **Business Feedback**: Business metrics and user satisfaction data
- **Security Feedback**: Security scan results and threat intelligence
- **Performance Feedback**: Performance trends and optimization opportunities

**Continuous Improvement**:
- **Pipeline Optimization**: Regular analysis of pipeline performance
- **Process Improvement**: Regular retrospectives and process updates
- **Tool Evaluation**: Regular evaluation of new tools and technologies
- **Metrics Review**: Regular review and refinement of metrics and alerts
- **Training and Knowledge Sharing**: Regular training on new processes and tools

Research current DevOps monitoring best practices and AWS observability services using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive pipeline monitoring with key DevOps metrics
- Quality metrics tracking and trending for continuous improvement
- Deployment health monitoring with automated validation
- Alerting and notification strategy for all pipeline events
- Observability integration providing end-to-end visibility

Key Deliverables

After completing the Deployment Pipeline exercises, you should have:

1. **CI/CD Pipeline Architecture**: Comprehensive automation with AWS native services
2. **Environment Management Strategy**: Multi-environment deployment with proper promotion gates
3. **Deployment Strategies**: Blue/green and canary deployments with automated rollback
4. **Quality Assurance Integration**: Comprehensive testing and quality gates
5. **Monitoring and Observability**: Complete visibility into pipeline and deployment health

Integration with Advanced Features

Your deployment pipeline leverages:

- **AWS Documentation MCP Server**: For current CI/CD and deployment best practices

- **Agent Steering**: For applying deployment standards and quality requirements

With comprehensive deployment pipeline specifications complete, you're ready to design the monitoring and observability infrastructure that will provide operational excellence for your chatbot application. The next section will focus on comprehensive monitoring, logging, and alerting strategies that ensure reliable operations and proactive issue detection.

Previous Next

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use **Cookie preferences**